Control Flow Analysis

CSE552 Program Analysis — Lecture 11

Jaemin Hong

Control Flow Analysis

- When functions are values, it is nontrivial to see which function is being called at each call site
- Control flow analysis conservatively approximates the interprocedural control flow (i.e., call graph)
- A call graph shows which functions may be called for each call site
- Call graphs provide a foundation for subsequently performing interprocedural dataflow analysis

Constraint Rules

For each program variable x, we introduce a constraint variable $[x]$ denoting the set of possible functions that x may refer to

- `fn x(...) { ... }`: $x \in [x]$
- `x = y`: $[y] \subseteq [x]$
- `x = y(z1, ..., zn)`:
  $\forall f.\, f \in [y] \Rightarrow ([z_1] \subseteq [a_f^1] \wedge \cdots \wedge [z_n] \subseteq [a_f^n] \wedge [\mathrm{RET}_f] \subseteq [x])$

Example 1 — Code and Constraints

```
fn foo() { ... }
fn bar() { ... }

if ... {
  x = foo;
} else {
  x = bar;
}
x();
```

Constraints:

$$\texttt{fn foo} : \mathit{foo} \in [\mathit{foo}]$$

$$\texttt{fn bar} : \mathit{bar} \in [\mathit{bar}]$$

$$\texttt{x = foo} : [\mathit{foo}] \subseteq [x]$$

$$\texttt{x = bar} : [\mathit{bar}] \subseteq [x]$$

Example 1 — Solution

$$[foo] = \{foo\}$$
$$[bar] = \{bar\}$$
$$[x] = \{foo, bar\}$$

Example 2 — Code and Constraints

```
fn foo(x) {
  RET_foo = x;
  return;
}
fn bar(y) {
  RET_bar = y;
  return;
}

z = foo;
w = z(bar);
```

Constraints:

$$\texttt{fn foo} : \mathit{foo} \in [\mathit{foo}]$$

$$\texttt{RET\_foo = x} : [x] \subseteq [\mathrm{RET}_{\mathit{foo}}]$$

$$\texttt{fn bar} : \mathit{bar} \in [\mathit{bar}]$$

$$\texttt{RET\_bar = y} : [y] \subseteq [\mathrm{RET}_{\mathit{bar}}]$$

$$\texttt{z = foo} : [\mathit{foo}] \subseteq [z]$$

$$\texttt{w = z(bar)} :$$

$$\mathit{foo} \in [z] \Rightarrow ([\mathit{bar}] \subseteq [x] \wedge [\mathrm{RET}_{\mathit{foo}}] \subseteq [w])$$

$$\wedge\ \mathit{bar} \in [z] \Rightarrow ([\mathit{bar}] \subseteq [y] \wedge [\mathrm{RET}_{\mathit{bar}}] \subseteq [w])$$

Example 2 — Solution

$$
\begin{aligned}
[foo] &= \{foo\} \\
[bar] &= \{bar\} \\
[x] &= \{bar\} \\
[\mathrm{RET}_{foo}] &= \{bar\} \\
[z] &= \{foo\} \\
[w] &= \{bar\}
\end{aligned}
$$

Cubic Algorithm

- The constraints for control flow analysis are an instance of a general class that can be solved in cubic time
- Many problems fall into this category

- A finite set of tokens $T = \{t_1, \ldots, t_k\}$
- A finite set of variables $V = \{x_1, \ldots, x_n\}$ whose values are sets of tokens
- Each constraint has one of the following forms:
  - $t \in x$
  - $x \subseteq y$
  - $t \in x \Rightarrow y \subseteq z$
- Goal: for a given collection of constraints, produce the least solution

Data Structures

- The algorithm maintains a directed graph where
  - Nodes correspond to constraint variables
  - Edges reflect inclusion constraints
- For each constraint variable x,
  - $x.\mathrm{sol} \subseteq T$ holds the solution for x
  - $x.\mathrm{succ} \subseteq V$ is the set of successors of x (i.e., the edges of the graph)
  - $x.\mathrm{cond}(t) \subseteq V \times V$ represents a set of conditional constraints for x and t
- We additionally have
  - $W \subseteq T \times V$ is a worklist
- All sets are initially empty

Helper Functions

We introduce helper functions for adding tokens and edges and propagating tokens

AddToken(t, x):
- **if** $t \notin x.sol$ **:**
  - x.sol.add(t)
  - W.add(t, x)

AddEdge(x, y):
- **if** $x \neq y \wedge y \notin x.succ$ **:**
  - x.succ.add(y)
  - **for** $t \in x.sol$ **:**
    - AddToken(t, y)

Propagate():
- **while** $W \neq \emptyset$ **:**
  - $(t, x) \leftarrow W$.removeOne()
  - **for** $(y, z) \in x.cond(t)$ **:**
    - AddEdge(y, z)
  - **for** $y \in x.succ$ **:**
    - AddToken(t, y)

Processing Constraints

We process each constraint using the helper functions

$t \in x$:
- AddToken(t, x)
- Propagate()

$y \subseteq z$:
- AddEdge(y, z)
- Propagate()

$t \in x \Rightarrow y \subseteq z$:
- **if** $t \in x.sol$ **:**
  - AddEdge(y, z)
  - Propagate()
- **else**
  - x.cond(t).add(y, z)

Cubic Algorithm — Example 1

Constraints: $t \in x$, $x \subseteq y$

- $t \in x$
  - AddToken(t, x): x.sol.add(t)
- $x \subseteq y$
  - AddEdge(x, y): x.succ.add(y)
  - $t \in x$.sol
  - AddToken(t, y): y.sol.add(t)

Cubic Algorithm — Example 2

Constraints: $x \subseteq y,\ t \in x$

- $x \subseteq y$
  - AddEdge(x, y): x.succ.add(y)
- $t \in x$
  - AddToken(t, x):
    - x.sol.add(t)
    - W.add(t, x)
  - Propagate():
    - $(t, x) = W$.removeOne()
    - $y \in x$.succ
    - AddToken(t, y): y.sol.add(t)

Cubic Algorithm — Example 3

Constraints: $t \in x$, $t \in x \Rightarrow y \subseteq z$

- $t \in x$
  - AddToken(t, x): x.sol.add(t)
- $t \in x \Rightarrow y \subseteq z$
  - AddEdge(y, z): y.succ.add(z)

Cubic Algorithm — Example 4

Constraints: $t \in x \Rightarrow y \subseteq z,\ t \in x$

- $t \in x \Rightarrow y \subseteq z$
  - $x.\text{cond}(t).\text{add}(y, z)$
- $t \in x$
  - AddToken(t, x): $W.\text{add}(t, x)$
  - Propagate():
    - $(t, x) = W.\text{removeOne}()$
    - $(y, z) \in x.\text{cond}(t)$
    - AddEdge(y, z): $y.\text{succ.add}(z)$

Time Complexity

Assumptions:

- Number of tokens $= O(n)$
- Number of variables $= O(n)$
- Number of $t \in x$ constraints $= O(n)$
- Number of $x \subseteq y$ constraints $= O(n^2)$

- Each pair (t, v) can be added at most once to the worklist, so the number of iterations of Propagate is $O(n^2)$
- AddEdge is called $O(n^2)$ times because there are $O(n^2)$ conditional constraints
- AddToken is called $O(n^3)$ times in total because there are $O(n^2)$ edges and $O(n)$ tokens
- Updating a set takes $O(1)$
- In total, the algorithm runs in $O(n^3)$ time

Possible Improvements

- Cycle elimination
- Maintaining the worklist in topological order
- Interleaving solution propagation and constraint processing
- Using shared bit vectors

Control Flow in Object-Oriented Languages

For each `x.m(...)`, we want to decide which method implementations may be called.

- Simple solution: select any method named *m* whose signature matches the argument types
- Better solution: class hierarchy analysis (CHA), which considers only the part of the class hierarchy spanned by the declared type of *x*
- More refined solution: rapid type analysis (RTA), which decides objects of which classes are actually instantiated

Summary

- Control flow analysis conservatively approximates call graphs when functions are first-class values
- Constraint rules relate each variable to the set of functions it may hold, including flow through calls and returns
- The constraint system is an instance of a general class solvable in cubic time via a worklist-based algorithm on a graph of inclusion edges
- Key data structures: per-variable solution sets, successor edges, and conditional constraints, driven by a token–variable worklist